

Foundations of Distributed Systems

Absolute Time, Concurrency, Logical Clocks, and Leader Election

Chapter 3.1 / 2025-09-26

Prof. Dr. Christian Tschudin

Dept of Mathematics and Computer Science, University of Basel

Communication and Coordination 3.1, Overview

- **Prelude: Coordinated Lunar Time (LTC)**
 - Global Positioning System (GPS)
 - Coordinated Universal Time (UTC), NTP
- Message Exchange and Concurrency
- Causality and Partial Order
- Logical Clocks, Vector Clocks, and the Git Commit Graph
- Leader Election

Mandate to create Coordinated Lunar Time (LTC)



EXECUTIVE OFFICE OF THE PRESIDENT
OFFICE OF SCIENCE AND TECHNOLOGY POLICY
WASHINGTON, D.C. 20502

April 2, 2024

MEMORANDUM FOR DEPARTMENTS AND AGENCIES PARTICIPATING IN THE WHITE
HOUSE **CISLUNAR TECHNOLOGY STRATEGY INTERAGENCY WORKING GROUP**

. . .

2. Policy Guidance

This policy requires federal agencies to develop celestial time standardization according to the following directions.

- a. **Coordinated Lunar Time (LTC) will act as the established standard to enable Cislunar operations and maintain traceability to UTC.**

The concept of Lunar Time is analogous to the ideal Terrestrial Time scale on Earth. Just as Terrestrial Time is set through an ensemble of atomic clocks on Earth, an ensemble of clocks on the Moon might set Lunar Time. This operational time standard, LTC, may directly employ or distribute the UTC offsets required to maintain both local time and UTC time within specific tolerances.

Why LTC?

<https://www.whitehouse.gov/wp-content/uploads/2024/04/Celestial-Time-Standardization-Policy.pdf>

- Lunar time differs from time on earth:
it runs $58.7 \mu sec$ faster, per earth day
- For lunar navigation, a GPS-like system is necessary;
these satellites must be synchronized among themselves;
not feasible to synchronize them with systems on earth

→ LTC will be an independent “time zone”

Global Positioning System (GPS)

- Operational since 1993. Main use is *positioning*, but many distributed systems use GPS as a *time source*
- GPS satellites synchronize their clocks, a base station corrects the satellite clocks.
- Constant correction is necessary: “satellite time” runs $38 \mu sec$ faster than earth time, per day.

GPS gives the illusion of global (earth) time, needs to be continuously corrected to do so. Accuracy: 10-40 nanoseconds

On the ground: sync of atomic clocks via radio transmissions

Universal Time Coordinated (UTC), UT, and many more

- Universal Time (UT, UT1, ...) based on Earth's rotation (astronomic time, → Greenwich Mean Time)
- Today's time pipeline:
 - TAI (International Atomic Time), using atomic clocks averaged over 200 atomic clocks, worldwide
 - UTC is “paper time” (calculated), has leap seconds
 - DUT (delta UT): predicted difference of UT with UTC
- UTC is timezone-independent, no daylight saving
“universal” only on earth, not lunar

<https://www.stjarnhimlen.se/comp/time.html> (*timescale jungle*)

Can we synchronize clocks?

Yes, but only within some accuracy interval

All coordination (in the universe) is via some message passing:

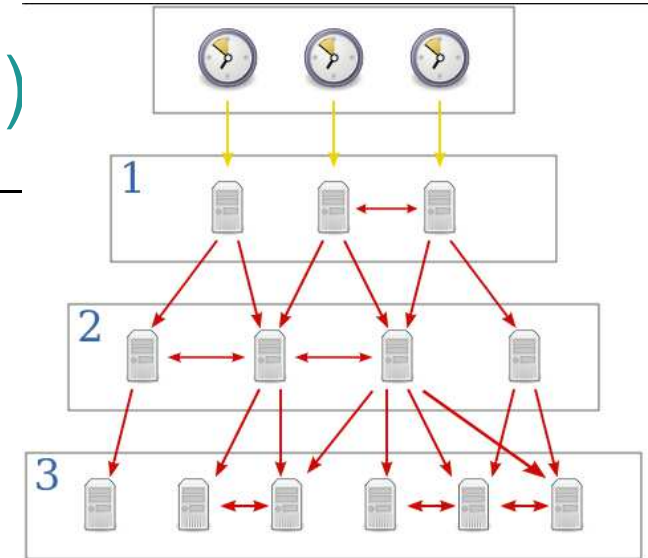
- matter:
 - molecules
 - letter
- signals
 - single photon
 - continuous electromagnetic wave

Transmission takes time and may vary, cannot be exactly known

Internet Time (Network Time Protocol)

Internet has own time delivery service.
Updates clock of *your* laptop!

- NTP propagates UTC, derived from ATI (atomic time), or other sources
- Several strata for fan-out:
 - sync (mutual adjustment) inside a stratum
 - precision decreases: nano (0), micro (1), to milliseconds
 - Android clock: can be off by seconds ...
- see <https://www.ntppool.org/en/>



No “global” time, in a precise way

In our real world, transmission time cannot be predicted (thermal noise).

How to know WHEN we sent, WHEN we received a message?

- neither sender nor receiver have access to some global (or anywhere in the universe) time instrument
- have to use messages to find out
- We *can* measure round-trip time (NTP does this), but this does not yield accurate time differences (as it includes imprecise transmission time)

Communication and Coordination 3.1, Overview

- Prelude: Coordinated Lunar Time (LTC)
 - Global Positioning System (GPS)
 - Coordinated Universal Time (UTC), NTP
- **Message Exchange and Concurrency**
- Causality and Partial Order
- Logical Clocks, Vector Clocks, and the Git Commit Graph
- Leader Election

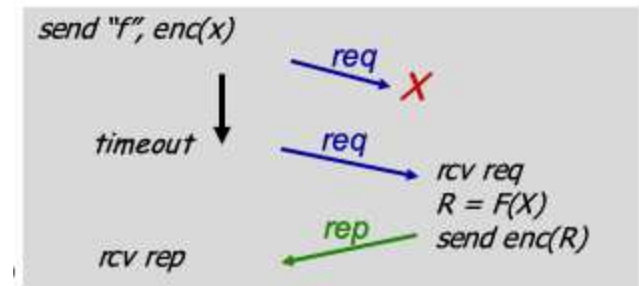
(Physical) Concurrency

Consequence of “no global time”:

- can't say for all event pairs which of the two events “happened before”
- such events are called **concurrent**

Real world vs modeling:

- We use message sequence charts (MSC) to show time progress, including message exchange
- so called “god view” of a system
- real systems cannot have or derive such a view



Logical Clocks

Although we have the arrow of physical time, time in distributed systems only leads to a partial ordering of events

- some events can't be ordered against each other:
neither $time(a) < time(b)$ nor $time(b) > time(a)$

Locally e.g., inside a node:

- total order is easy to obtain: we have sequential CPUs, or a single bus (which serializes access)
- nodes can maintain an internal counter, a “logical clock”

Logical Clocks (contd)

From local ordering (total) to global ordering (partial):

- inside a node: local counter
- between nodes: logical ordering based on causality
 - use “send-happens-before-receive” causality (cause-comes-before-effect)
- creates a happens-before relation *for some events*

A minimal logical clock:

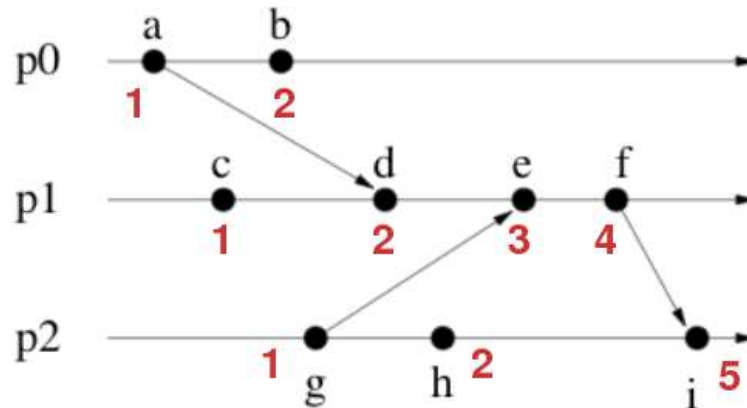
- nodes have a local counter, monotonically growing
- increase value for each send event, for each reception event

Comparing Logical Clock Values

We want to assign integer values to events such that

if $a \rightarrow b$, then $time(a) < time(b)$

- works when comparing *local* clock values
- but not for comparing clock values across nodes:
 $time(a) < time(b)$ does not imply $a \rightarrow b$



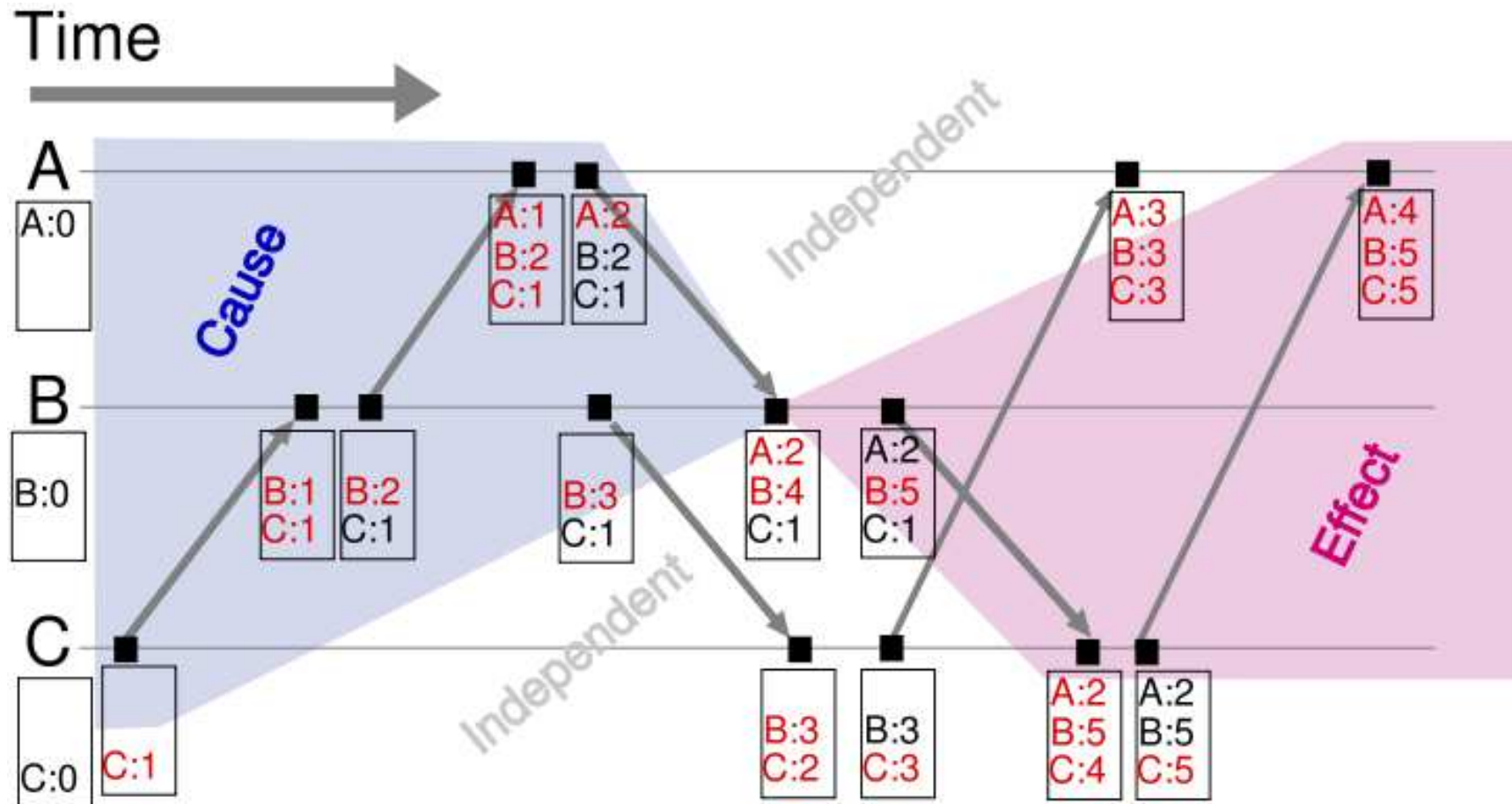
Problem when comparing events *g* and *b*. Can we do better?

Vector Clocks (Mattern, 1989)

- Timestamp = **vector of logical clocks**, attached to each msg
- Before send: increment local clock, respective component
- On msg reception, update local vector:
 - merge with local vector, using max of component values
 - increment local clock (and respective vector component)
- We now define $<$ (before-than) for two timestamps as:
all components must be equal or less, and then
at least one component must be different.

Incomparable values meaning “concurrent”! See causality figure.

Cause and Effect: preceeding or independent/concurrent

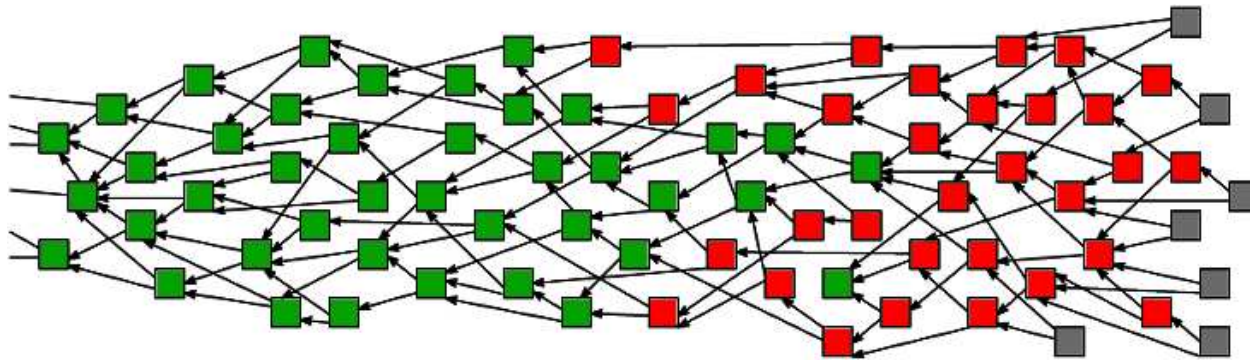


Vector Clocks and DAGs

Vector clock as one way to capture causality:

- Mattern's solution uses a vector, nodes must be known in advance
- other solutions possible, based on causality graphs

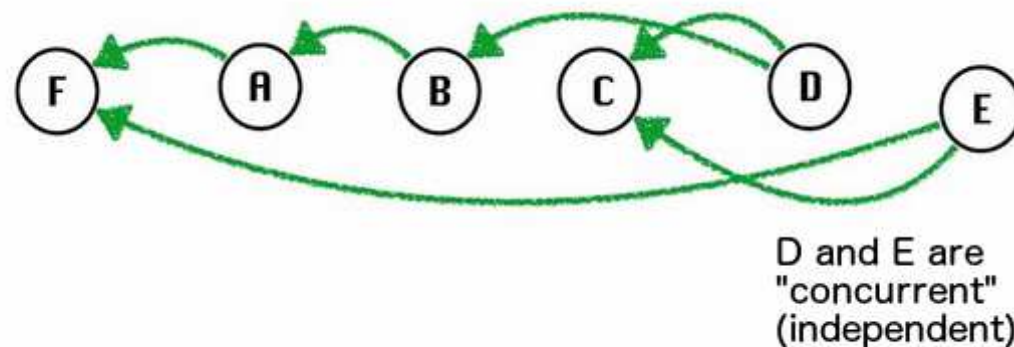
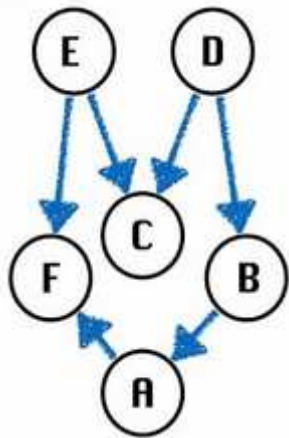
Tangle data struct – captures causality via hashing



- “*You are a predecessor (event) because I computed your hash value*”. Hashing shown as pointer, gray nodes = “tip nodes”

Vector Clocks and DAGs (contd)

- DAG used by Git: each commit action is a new event node with one predecessor, “merge” actions point to several commits
- DAG also used by IOTA crypto currency (instead of a global blockchain, which establishes a single global logical clock value)
- Relates to $\rightarrow$ *topological sort*, serialization of a DAG



Google's Spanner (2012)

Should all distributed systems use vector clocks, causality graphs?
Benefit from “almost accurate time” as much as possible:

- use physical time, delivered by **GPS**,
as “global clock”, although it's approximated
- time values are associated with a precision interval.
Applications must handle overlapping intervals.
Use case: independence of database transactions

GPS timestamping also used for logging in network experiments
(synchronize/align log files), even to pace TCP sessions!

Solving Tasks in a Distributed Systems (Intro)

Characterization of asynchronouse communication:

- nodes interconnected through a network
- loosely coupled (message exchange)
- independent activity (concurrency)
- unreliable (lossy channels, node failures)

It's hard to write reliable distributed programs.

- Many impossibility results, too!
- Even “autonomics” can not help, then.

Need for a Coordinator

Hypothetical task: keep a system-wide counter.

How to cope with node crashes, network partitioning?

- Easiest is to have a single “coordinator” (server), all other nodes in the net behave as clients.
- Not relevant *which* node is coordinator.
- In case of server crash: pick another node.
- In case of network partition: add a 2nd server.

“Leader election” needed. In general: a consensus finding problem

Communication and Coordination 3.1, Overview

- Prelude: Coordinated Lunar Time (LTC)
 - Global Positioning System (GPS)
 - Coordinated Universal Time (UTC), NTP
- Message Exchange and Concurrency
- Causality and Partial Order
- Logical Clocks, Vector Clocks, and the Git Commit Graph
- **Leader Election**

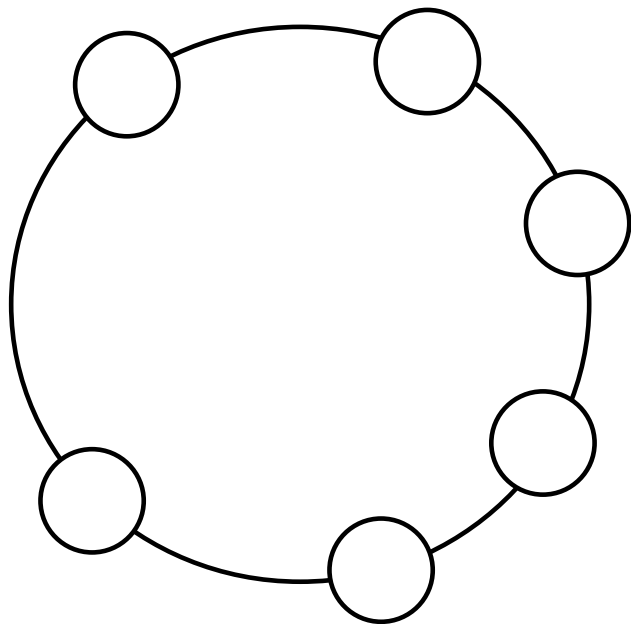
Leader Election (LE)

Problem description:

- Nodes have status, with one “elected” (winning) and “not-elected” (loosing) state, as well as other states.
- After being elected, a node stays elected (same for not-elected)
- Required termination properties of a LE algorithm:
 - eventually, **every node** has either won or lost
 - **exactly one node enters a winning state.**

Topology – Ring

In proofs for distributed systems, a ring topology is often used instead of arbitrary graphs.



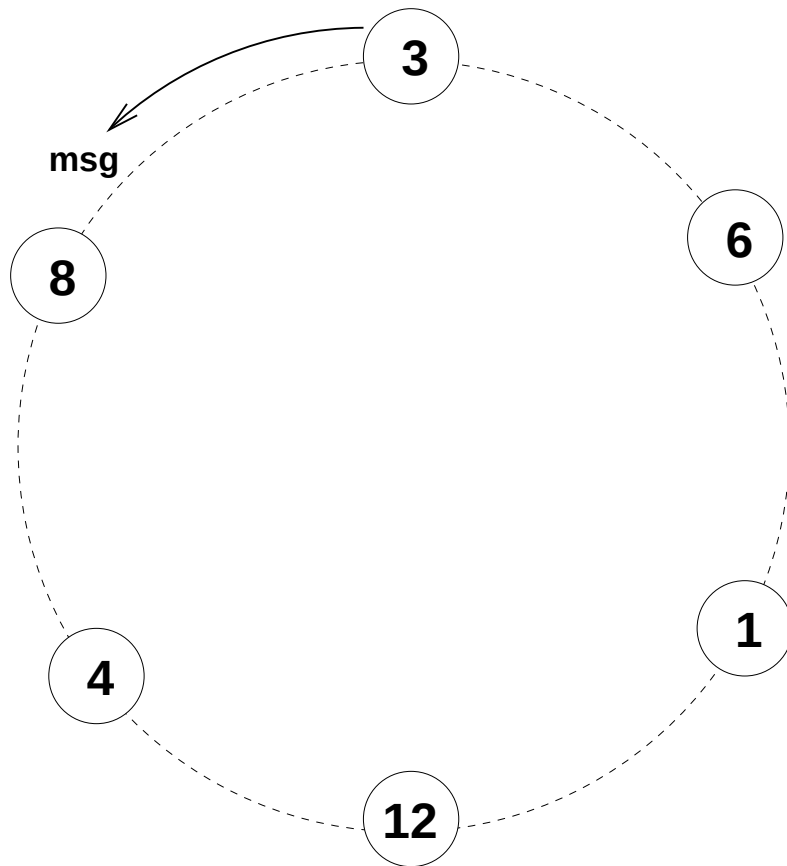
- Ring looks like a toy case, but is complex enough for some impossibility results.
- More variants of ring topologies:
 - **anonymous ring** vs
 - nodes having a unique identifier
 - uniform / non-uniform algos ...
- etc

Leader Election: Theory Results

- There is **no** leader election algorithm for anonymous rings (no IDs), even if:
 - algorithm knows the ring size
 - we use the simpler synchronous comm. model (see later)
- Proof sketch:
 - consider config with all proc in same start state
 - all will send same message to neighbor
 - all will receive same message, do same transition
 - all will enter same “win” or “lose” state: violation.

Theorem: There exist LE algorithms when nodes have unique IDs.

Leader Election Algorithm in a Ring with IDs



- All nodes have identical code, but unique IDs
- How to pick a winner?
- How to make sure that everybody agrees on the winner?

Leader Election Algorithm (Le Lann, 1977)

- send value of own ID to the left
 - when receiving an id j (from the right):
 - o if $j > \text{ID}$ then forward to the left (we loose)
 - o if $j = \text{ID}$ then elect self (we win)
 - o if $j < \text{ID}$ then do nothing
-
- Intuitive: IDs traveling around, filters max value
 - Works in asynchronous and synchronous model
 - Above is not optimal: $O(n^2)$ messages (and n time steps)
 - Not possible to do better than $O(n \log n)$ in asynch model

Discussion on Coordination

- Leader Election (as shown above) is simplified:
 - no message loss
 - all processors start at the same time
 - no processor fails
 - no processor cheats
 - special ring topology

“Extrema finding”:

- also key element in Garcia-Molinas “Bully Algorithm” (1982)
- but now for arbitrary topologies?

Leader Election: Constructing a Spanning Tree

“Prepare the network” before the election (logical ring, spanning tree)

- In a static graph (theorem):
if all nodes have unique IDs, a single *minimum spanning tree* (MST) can be constructed. (All nodes agree on this tree, even if the construction is started at arbitrary places)
- Once a MST is available, do “scatter/gather”:
 - send out LE request, scatter-wave goes to the leaf nodes,
 - elect (somehow) leader in each subtree,
 - elect next-level-leader as gather-wave travels towards the root.

What about wireless?

Leader Election: MANET and Flooding

Wireless networks permit spanning tree construction using flooding:

- All nodes can initiate a “Campaign-For-Leader” (CFL message)
- Several CFL waves can coexist,
 - waves are tagged with a timestamp,
 - the timestamp with the lowest timestamp eventually wins (this is again “extrema-finding”)
- Although not minimal, this tree can be used for leader election (or simply: the root is the leader).

Many complex scenarios to consider. “Leader Election” is not simple!

LE-related Concepts Introduced Above

- cheat-proof algorithm
(cheating as a “failure mode”)
- synchronous/asynchronous distributed system
- weakly self-stabilizing algorithm

Some of these concepts will be explored in this lecture series.